

Spark API-Optionen – Referenz

Beim Lesen und Schreiben von Daten mit Spark (z. B. `spark.read...` oder `spark.readStream...`) gibt es zahlreiche Zusatz-Optionen, mit denen man das Verhalten steuern kann – etwa welches Dateiformat gelesen wird, wie mit fehlerhaften Zeilen umgegangen wird oder wie Streaming-Quellen überwacht werden. Diese Seite ist eine Nachschlage-Referenz für genau diese Optionen und richtet sich vor allem an Personen, die bereits mit den Grundlagen von Spark vertraut sind und gezielt nach einer bestimmten Option suchen.

Zwei große Kategorien

Die Optionen lassen sich grob in zwei Bereiche einteilen: Optionen für den **DataFrameReader** (klassisches, einmaliges Einlesen von Daten – "Batch") und Optionen für den **DataStreamReader** (kontinuierliches Einlesen von Datenströmen – "Streaming").

DataFrameReader – Optionen für Batch-Verarbeitung

Einige Optionen gelten unabhängig vom Dateiformat, zum Beispiel:

- `ignoreCorruptFiles` / `ignoreMissingFiles` – beschädigte bzw. fehlende Dateien beim Einlesen ignorieren
- `modifiedAfter` / `modifiedBefore` – nur Dateien innerhalb eines bestimmten Änderungszeitraums einlesen
- `pathGlobFilter` – nur Dateien einlesen, die einem bestimmten Namensmuster entsprechen
- `recursiveFileLookup` – auch in Unterordnern nach Dateien suchen
- `ignoredPathSegmentRegex` – bestimmte Pfad-Bestandteile beim Einlesen ausschließen

Zusätzlich gibt es für jedes Dateiformat eigene, spezifische Optionen:

Format	Wichtige Optionen (Auswahl)
Avro	<code>avroSchema</code> , <code>mergeSchema</code> , <code>mode</code> , <code>rescuedDataColumn</code> , <code>recursiveFieldMaxDepth</code>
CSV	<code>header</code> , <code>delimiter</code> / <code>sep</code> , <code>encoding</code> , <code>quote</code> , <code>multiline</code> , <code>inferSchema</code> , <code>mode</code> , <code>rescuedDataColumn</code>
Excel	<code>dataAddress</code> , <code>headerRows</code> , <code>operation</code> (readSheet / listSheets)

JSON	<code>multiLine</code> , <code>mode</code> , <code>inferTimestamp</code> , <code>rescuedDataColumn</code> , <code>singleVariantColumn</code>
Kafka	<code>endingOffsets</code> , <code>endingOffsetsByTimestamp</code> , <code>endingTimestamp</code>
Parquet	<code>mergeSchema</code> , <code>datetimeRebaseMode</code> , <code>rescuedDataColumn</code>
XML	<code>rowTag</code> , <code>mode</code> , <code>inferSchema</code> , <code>attributePrefix</code> , <code>rescuedDataColumn</code>
ORC / Text	<code>mergeSchema</code> , <code>encoding</code> , <code>wholeText</code>

Beispiel für das Einlesen von JSON-Dateien mit mehrzeiligen Datensätzen:

```
df = spark.read.format("json").option("multiLine", True).load("/path/to/data")
```

DataStreamReader – Optionen für Streaming

Für Streaming-Quellen gibt es allgemeine Optionen wie:

- `maxBytesPerTrigger` / `maxFilesPerTrigger` – Begrenzung, wie viele Daten pro Verarbeitungsschritt (Trigger) gelesen werden
- `maxFileAge` – wie alt eine Datei maximal sein darf, um noch berücksichtigt zu werden
- `cleanSource` / `latestFirst` – Aufräumverhalten bzw. Verarbeitungsreihenfolge der Quelldateien
- `sourceArchiveDir` – Zielordner zum Archivieren bereits verarbeiteter Dateien

Auto Loader (cloudFiles)

Für komplexere Ingestion-Szenarien (das fortlaufende Einlesen neuer Dateien aus Cloud-Speicher) empfiehlt Databricks den sogenannten **Auto Loader**, erkennbar am Format `cloudFiles`.

Wichtige Optionen dafür:

- `cloudFiles.format` – Pflichtangabe, welches Dateiformat gelesen wird
- `cloudFiles.schemaLocation` / `cloudFiles.schemaEvolutionMode` – wo das erkannte Schema gespeichert wird und wie mit Schemaänderungen umgegangen wird
- `cloudFiles.maxFilesPerTrigger` / `cloudFiles.maxBytesPerTrigger` – Mengenbegrenzung pro Verarbeitungsschritt
- `cloudFiles.cleanSource` – automatisches Aufräumen bereits verarbeiteter Dateien (Modi: OFF, DELETE, MOVE, mit einstellbarer Aufbewahrungsdauer; standardmäßig nach 30 Tagen)

- `cloudFiles.partitionColumns` / `cloudFiles.includeExistingFiles` – Partitionierung und ob bereits vorhandene Dateien mit eingelesen werden
- `cloudFiles.inferColumnTypes` / `cloudFiles.schemaHints` – automatische Typerkennung bzw. manuelle Hinweise zum Schema
- `cloudFiles.backfillInterval` / `cloudFiles.allowOverwrites` – regelmäßiges Nachholen verpasster Dateien bzw. Umgang mit überschriebenen Dateien

State-Store-Optionen (für Structured Streaming)

Für den internen Zustandsspeicher (State Store) von Structured-Streaming-Abfragen gibt es zusätzliche, spezialisierte Optionen, unter anderem `batchId`, `operatorId`, `storeName`, `joinSide`, sowie Optionen zum Lesen von Änderungsverläufen wie `readChangeFeed`, `changeStartBatchId`, `changeEndBatchId`, `snapshotStartBatchId`, `snapshotPartitionId`, `stateVarName`, `readRegisteredTimers` und `flattenCollectionTypes`. Diese Optionen sind vor allem für fortgeschrittenes Debugging und die Analyse des internen Streaming-Zustands relevant.

Wichtige Querschnittskonzepte

Rescued Data Column

Ein Mechanismus, der Daten „rettet“, die nicht zum erwarteten Schema oder Datentyp passen, statt sie einfach zu verwerfen. Auto Loader aktiviert diesen Mechanismus standardmäßig. Wichtig zu wissen: Dieses Feature steht im veralteten `COPY INTO`-Befehl nicht zur Verfügung.

Parser-Modi

Beim Einlesen fehlerhafter Daten gibt es drei Verhaltensweisen, gesteuert über die Option `mode`:

- **FAILFAST** – bricht die Verarbeitung mit einem Fehler ab
- **PERMISSIVE** – setzt fehlerhafte Werte auf `null` und macht weiter
- **DROPMALFORMED** – verwirft die betroffene Zeile komplett

Datums- und Zeit-Parsing

Für Datums- und Zeitwerte lassen sich verschiedene Formate über eigene Optionen konfigurieren, etwa `dateFormat`, `timestampFormat`, `timestampNTZFormat` und `timeZone`.

Schema-Evolution

Über `cloudFiles.schemaEvolutionMode` lässt sich steuern, wie Auto Loader reagiert, wenn in neuen Dateien zusätzliche Spalten auftauchen, die im bisherigen Schema noch nicht bekannt waren – zum Beispiel automatische Anpassung des Schemas oder Ablehnen der Datei.

Wichtige Praxishinweise

- Für komplexe Ingestion-Szenarien wird grundsätzlich Auto Loader statt einfacher Batch-Reads empfohlen.
- Manche Features setzen mindestens Databricks Runtime 11.3 LTS voraus.
- Die Dokumentation ist auch in weiteren Sprachen verfügbar (Englisch, Japanisch, Portugiesisch, Französisch).

Quelle: <https://docs.databricks.com/aws/en/spark/api-options>

Stand: 2026-08-06